

OPERATING SYSTEMS
UE25CS242A

Introduction to Threads, System calls for Inter Process Communication

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. "Operating System Concepts", Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. "Advanced Programming in the Unix Environment", Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from "Operating Systems, Internals and Design Principles", William Stallings, 9th Edition, Pearson, 2018.

PES UNIVERSITY
CELEBRATING 50 YEARS

Operating Systems

System calls for Inter Process Communication

Dr. Anitha K
Department of Computer Science and Engineering

Pipes

- Pipes are the oldest form of UNIX System IPC and are provided by all UNIX systems.
- Pipes have two limitations.
  1. Historically, they have been half duplex (i.e., data flows in only one direction). Some systems now provide full-duplex pipes, but for maximum portability, we should never assume that this is the case.
  2. Pipes can be used only between processes that have a **common ancestor**. Normally, a pipe is created by a process, that process calls fork, and the pipe is used between the parent and the child.

pipe()

- A pipe is created by calling the **pipe()** function.

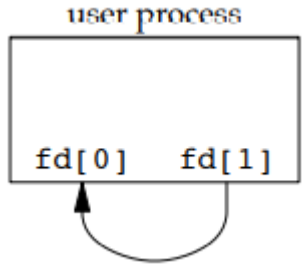
```
#include <unistd.h>
int pipe(int fd[2]);
```

- Return value: 0 if OK, −1 on error
- Two file descriptors are returned through the fd argument:
  1. fd[0] is open for reading
  2. fd[1] is open for writing.
- The output of fd[1] is the input for fd[0]

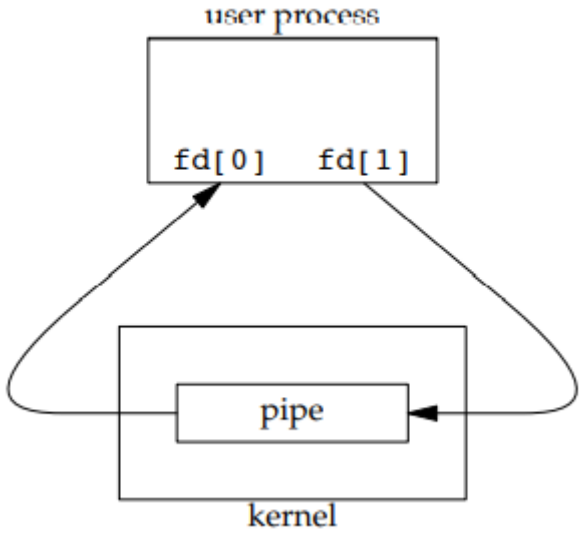
pipe()

- Two ways to picture a half-duplex pipe

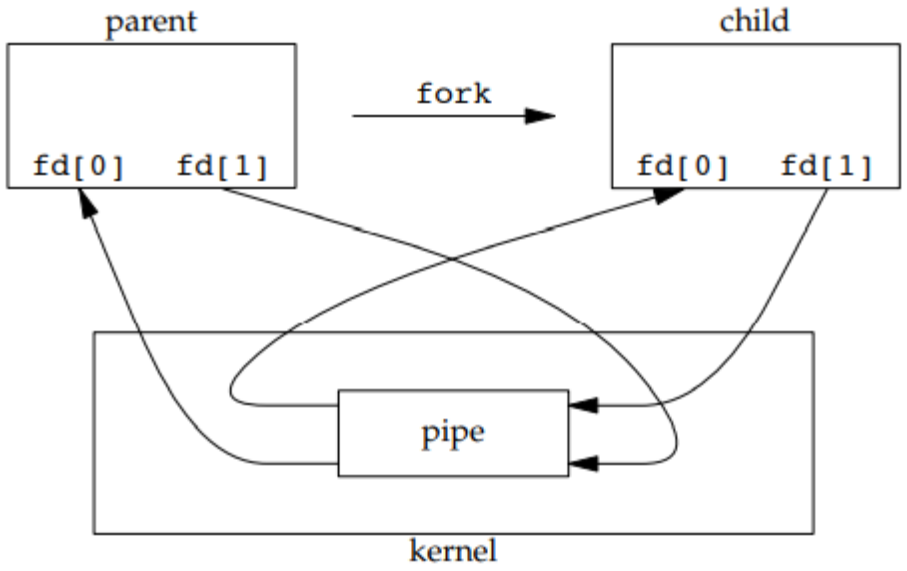
1. The two ends of the pipe connected in a single proce



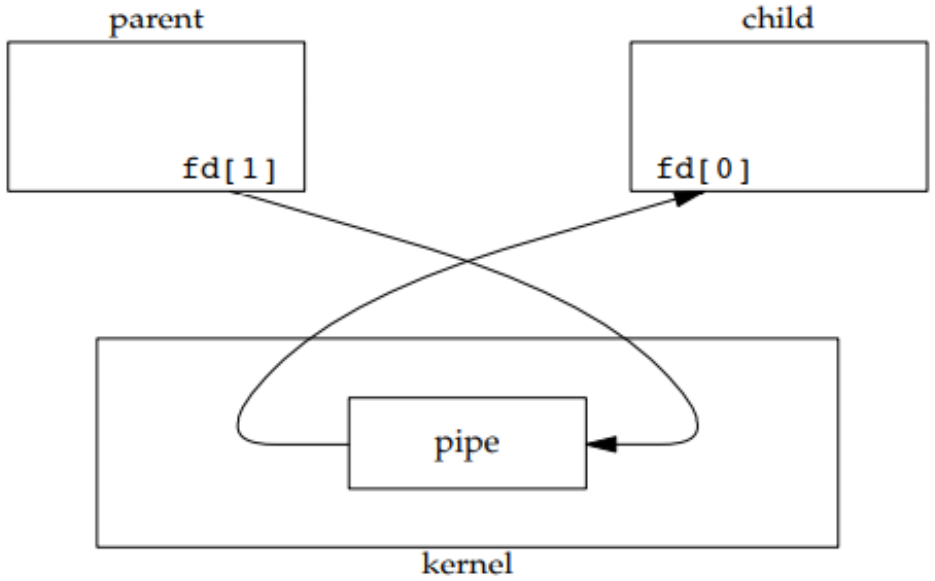
1. The data in the pipe flows through the kernel



pipe()



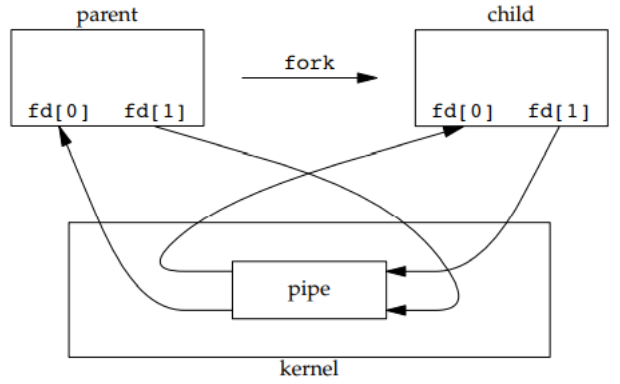
Half-duplex pipe after a fork



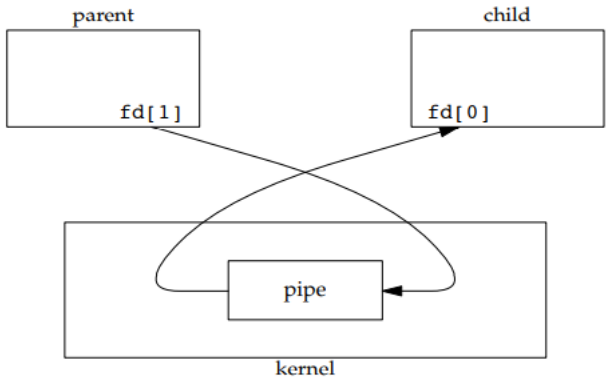
Pipe from parent to child

pipe()

- Normally, a process that calls pipe then calls fork, creating an IPC channel from the parent to the child or vice versa.



- For a pipe from the parent to the child, the parent closes the read end of the pipe (fd[0]) and the child closes the write end (fd[1]).
- For a pipe from the child to the parent, the parent closes fd[1], and the child closes fd[0].



Pipe from parent to child

popen() and pclose()

- A common operation is to create a pipe to another process to either read its output or send it input, the standard I/O library has historically provided the **popen()** and **pclose()** functions.
- These two functions handle all the work: creating a pipe, forking a child, closing the unused ends of the pipe, executing a shell to run the command, and waiting for the command to terminate.

```
#include <stdio.h>

FILE *popen(const char *cmdstring, const char *type);


int pclose(FILE *fp);
```

- The function **popen()** does a fork and exec to execute the cmdstring and returns a standard I/O file pointer.
- If type is "r", the file pointer is connected to the standard output of cmdstring.
- If type is "w", the file pointer is connected to the standard input of cmdstring.
- The function popen() returns file pointer if OK, NULL on error.
- The function **pclose()** closes the standard I/O stream, waits for the command to terminate, and returns the termination status of the shell or -1 on error.

OPERATING SYSTEMS

FIFOs

- FIFOs are sometimes called **named pipes**.
- Unnamed pipes can be used only between related processes when a common ancestor has created the pipe.
- With FIFOs, unrelated processes can exchange data.
- Creating a FIFO is similar to creating a file

```
#include <sys/stat.h>
int mkfifo(const char *path, mode_t mode);
int mkfifoat(int fd, const char *path, mode_t mode);
```

- Functions mkfifo() and mkfifoat() return 0 if OK, −1 on error.
- Once a FIFO has been created using mkfifo() or mkfifoat() , it can be opened using open(). Normal file I/O functions (e.g., close, read, write, unlink) all work with FIFOs.
- There are two uses for FIFOs.
  1. FIFOs are used by shell commands to pass data from one shell pipeline to another without creating intermediate temporary files.
  2. FIFOs are used as rendezvous points in client–server applications to pass data between the clients and the servers.

Message Queues

- A message queue is a linked list of messages stored within the kernel and identified by a message queue identifier.
- A new queue is created or an existing queue opened by **msgget**.

```
#include <sys/msg.h>
int msgget(key_t key, int flag);
```

- Returns message queue ID if OK, −1 on error.
- New messages are added to the end of a queue by **msgsnd**.

```
#include <sys/msg.h>
int msgsnd(int msqid, const void *ptr, size_t nbytes, int flag);
```

- Every message has a positive long integer type field, a non-negative length, and the actual data bytes all of which are specified to **msgsnd** when the message is added to a queue.
- Messages are fetched from a queue by **msgrcv**.

```
#include <sys/msg.h>
ssize_t msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);
```

- Messages need not be fetched in a first-in, first-out order. Instead, can be fetched based on their type field.

Semaphores

- A **semaphore** is a counter used to provide access to a shared data object for multiple processes.
- To obtain a shared resource, a process needs to do the following:
  1. Test the semaphore that controls the resource.
  2. If the value of the semaphore is positive, the process can use the resource. In this case, the process decrements the semaphore value by 1, indicating that it has used one unit of the resource.
  3. Otherwise, if the value of the semaphore is 0, the process goes to sleep until the semaphore value is greater than 0. When the process wakes up, it returns to step 1.
- When a process is done with a shared resource that is controlled by a semaphore, the semaphore value is incremented by 1.
- If any other processes are asleep, waiting for the semaphore, they are awakened.

Semaphores

- First need to obtain a semaphore ID by calling the semget function.

```
#include <sys/sem.h>
int semget(key_t key, int nsems, int flag);
```

- Returns: semaphore ID if OK, −1 on error.
- **semctl** function is the catchall for various semaphore operations.

```
#include <sys/sem.h>
int semctl(int semid, int semnum, int cmd, ... /* union semun arg */ );
```

- The function **semop** atomically performs an array of operations on a semaphore set.

```
#include <sys/sem.h>
int semop(int semid, struct sembuf semoparray[], size_t nops);
```

- The semoparray argument is a pointer to an array of semaphore operations.
- Returns 0 if OK, −1 on error.

Linux Domain Sockets

- A Linux Domain Socket is an IPC mechanism that enables processes on the same machine to communicate using the socket API without using the network stack.
- **Socket Address Family**
- Linux Domain Sockets use the **AF_UNIX (or AF_LOCAL)** address family.

```
socket(AF_UNIX, SOCK_STREAM, 0);
```

Socket Address Structure

- The address for a Linux Domain Socket is represented using the ***sockaddr_un*** structure.

```
struct sockaddr_un
                    {
                     sa_family_t sun_family;
                     char sun_path[108];
                };
```

Linux Domain Sockets

Working of the Server Program

- Step 1: Create the Socket

```
server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
```

- Step 2: Configure the Socket Address

```
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, "/tmp/mysocket");
```

- Step 3: Remove Any Existing Socket File

```
unlink("/tmp/mysocket");
```

- Step 4: Bind the Socket

```
bind(server_fd,(struct sockaddr *)&addr, sizeof(addr));
```

Linux Domain Sockets

Step 5: Listen for Connections

```
listen(server_fd, 5);
```

Step 6: Accept the Client Connection

```
client_fd = accept(server_fd, NULL, NULL);
```

Step 7: Receive Data

```
read(client_fd, buffer, sizeof(buffer));
```

Step 8: Send a Reply

```
write(client_fd, "Hello Client", 13);
```

Step 9: Close the Connection

```
close(client_fd);
close(server_fd);
unlink("/tmp/mysocket");
```

Linux Domain Sockets

Working of the Client

- Step 1: Create the Socket

```
sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
```

- Step 2: Specify the Server Address

```
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, "/tmp/mysocket");
```

- Step 3: Connect to the Server

```
connect(sockfd,(struct sockaddr *)&addr, sizeof(addr));
```

- Step 4: Send Data

```
write(sockfd, "Hello Server", 13);
```

Linux Domain Sockets

- Step 5: Receive the Reply

```
read(sockfd, buffer, sizeof(buffer));
```

- Step 6: Close the Socket

```
close(sockfd);
```

OPERATING SYSTEMS

Threads and Concurrency

Dr. Anitha K
Department of Computer Science and Engineering

Overview and Motivation

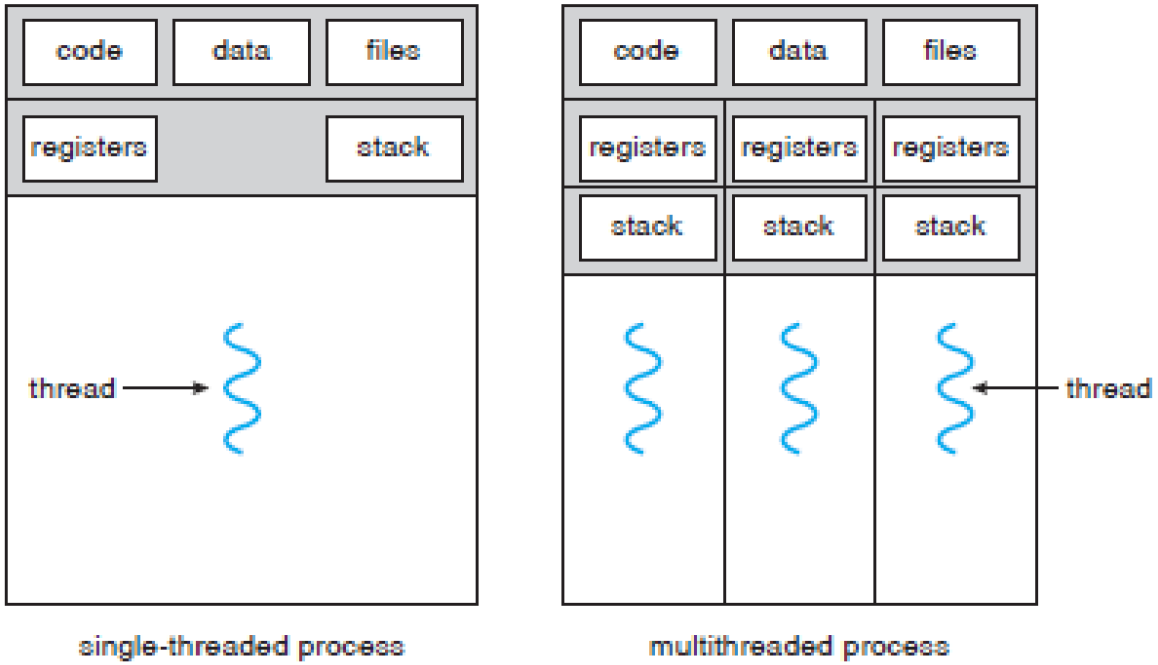
- A Thread is a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems.
- It consists of thread ID, Program counter, a register set and stack.
- Shares with other threads of same process its code, data,file descriptors, signals.
- Most modern applications are multithreaded.
- Threads run within application.
- Multiple tasks with the application can be implemented by separate threads.
- Application 1: internet browser.
  - numerous tabs open at a given time.
  - Multiple threads of execution are used to load content, display animations, play a video, fetch data from a network and so on.

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Single threaded and multithreaded processes

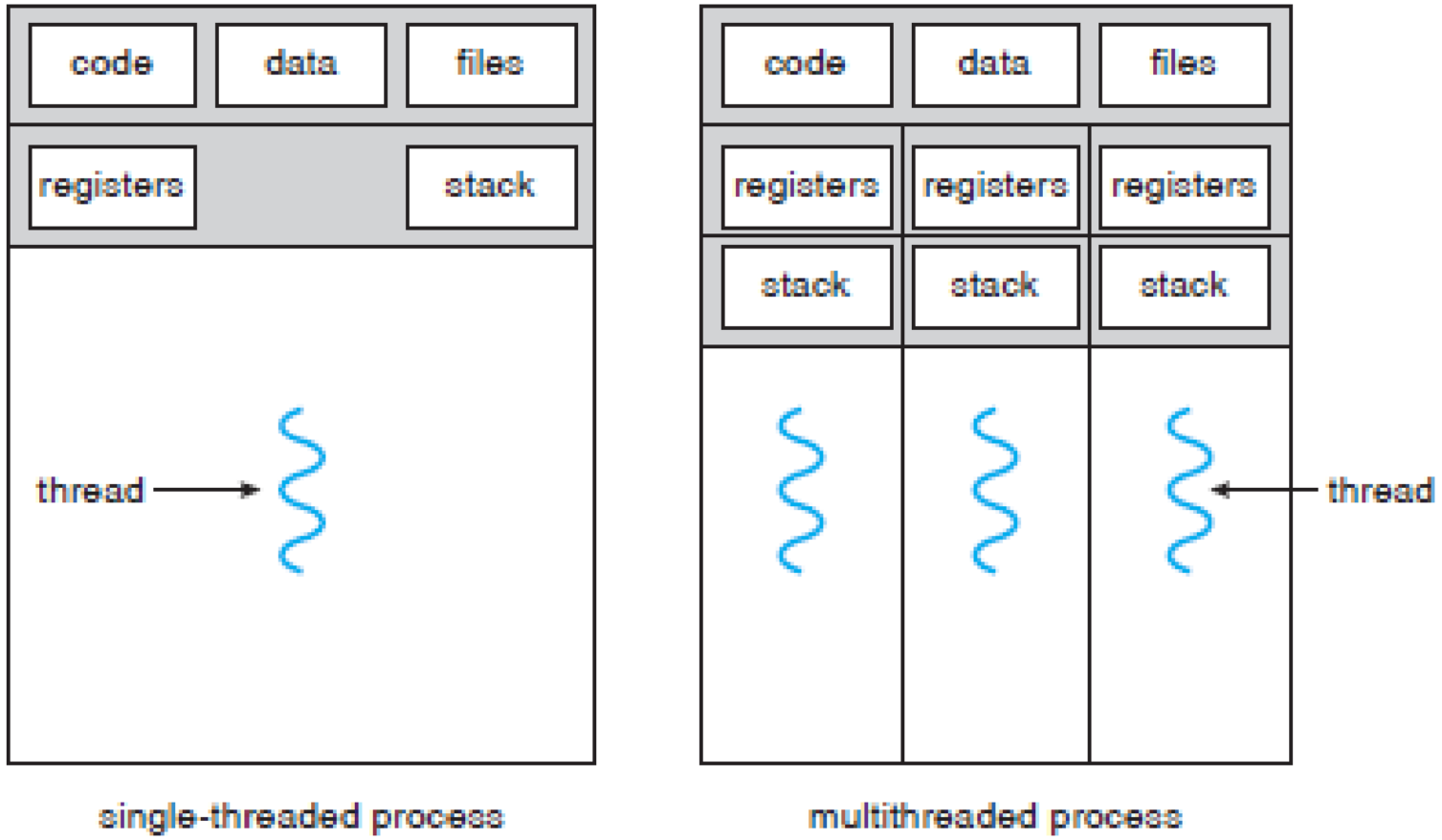
- Application 2: Word processor
  - Thread to accept input
  - A thread for spell checking
  - A thread for grammar checking



single-threaded process multithreaded process

OPERATING SYSTEMS

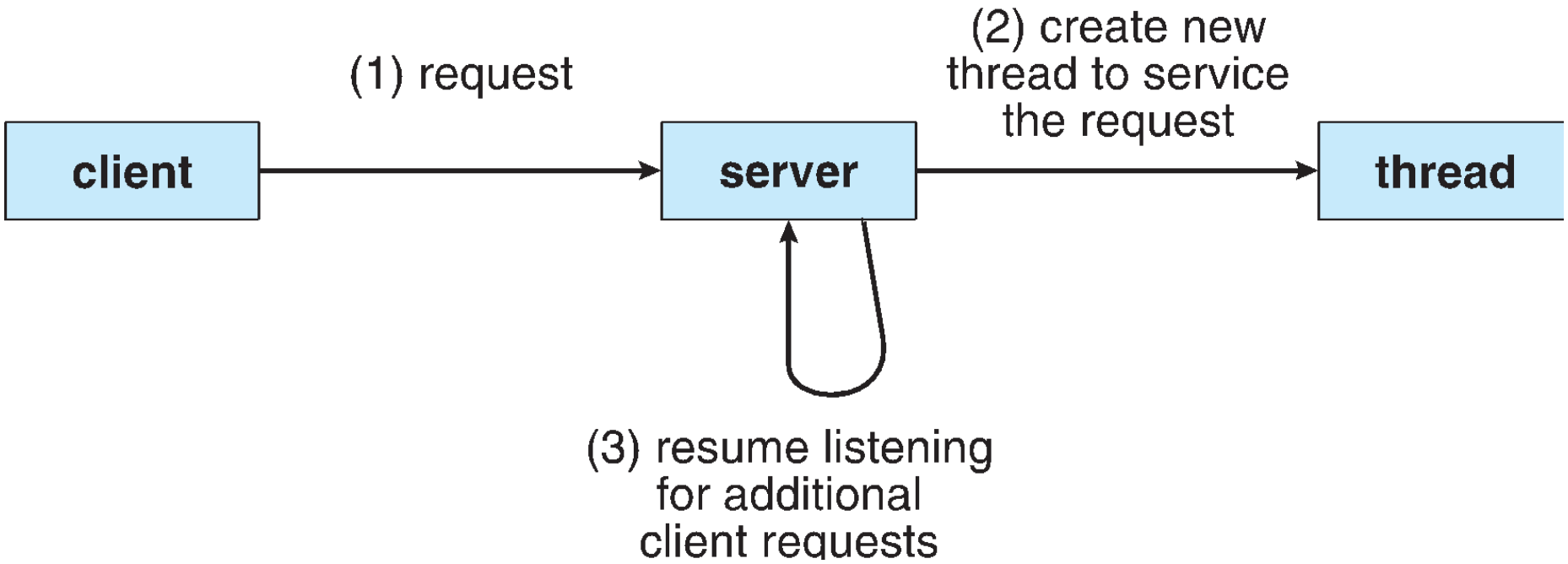
Single threaded and multithreaded processes



single-threaded process

multithreaded process

Multithreaded Server Architecture



Benefits

- Responsiveness
- Resource Sharing
- Economy
- Scalability

PES UNIVERSITY
CELEBRATING 50 YEARS

Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked or performing lengthy operations.
  - It is useful in designing user interfaces.
  - A multithreaded Web browser could allow user interaction in one thread while an image was being loaded in another thread.
  - Example: click on the button
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing.
  - Programmer needs to specify the techniques for sharing.
  - But threads share the memory and other resources.
  - Sharing of resources helps in having many threads within the same address space.

PES UNIVERSITY
CELEBRATING 50 YEARS

Benefits

- **Economy –** cheaper than process creation, thread switching lower overhead than context switching.
  - In Solaris, for example, creating a process is about thirty times slower than creating a thread, and context switching is about five times slower.
- **Scalability –** process can take advantage of multiprocessor architectures.
  - Threads can run on multiple cores parallelly.

PES UNIVERSITY
CELEBRATING 50 YEARS

Processes vs Threads



Process

- Will by default not share memory
- Most file descriptors not shared
- Don't share filesystem context
- Don't share signal handling

Thread

- Will by default share memory
- Will share file descriptors
- Will share filesystem context
- Will share signal handling

Thread Concepts



- A thread consists of the following information necessary to represent an execution context within a process.
- Thread ID that identifies the thread within a process
- Every thread has-
  - a thread ID
  - set of register values
  - stack
  - scheduling priority and policy
  - signal mask
  - ErrNo variable
  - Thread-specific data

- When a thread is created, there is no guarantee which will run first: the newly created thread or the calling thread.

Attributes Shared



Attributes shared by Threads

- process ID and parent process ID;process group ID and session ID;
- controlling terminal;
- process credentials (user and group Ids);open file descriptors;

Attributes specific to Threads

- thread ID ;signal mask;
- Thread-specific data ;
- the errno variable;



THANK YOU

Dr. Anitha K
Department of Computer Science and Engineering
anithak@pes.edu